Gestion des Erreurs

---

1. La Philosophie Go : *"Errors are values"* (Zéro Exception)

En C++, Java, C# ou Python, la gestion des erreurs repose sur le mécanisme des exceptions (`try / catch / throw`). Les créateurs de Go ont volontairement **banni les exceptions** pour trois raisons d'ingénierie logicielle :

1. **Flux de contrôle invisible :** Avec `throw`, n'importe quel appel de fonction peut interrompre brutalement l'exécution et remonter 15 niveaux de pile sans que la signature de la fonction ne l'indique.
2. **Coût d'exécution du *Stack Unwinding* :** Construire une *Stack Trace* complète à chaque erreur attendue (ex: un fichier introuvable ou un timeout réseau) est très coûteux en CPU et en mémoire.
3. **Négligence du développeur :** Le `try / catch` encourage à englober 50 lignes de code dans un seul bloc `try` en traitant toutes les pannes de manière générique.

1.1 Le retour au pragmatisme du C, la sécurité en plus

En langage C, on vérifie les erreurs via le code retour entier (`int fd = open(...); if (fd == -1)`), mais cela oblige à utiliser la variable globale `errno` ou à écraser la valeur de retour métier. En Go, grâce aux **retours multiples**, une fonction qui peut échouer retourne à la fois son résultat métier **et** une valeur d'erreur en dernière position :

```go
resultat, err := operationRisquee()
if err != nil {
	// Traitement explicite et immédiat de l'erreur !
	return err
}
// Ici, on a la garantie absolue que 'resultat' est valide (Flux "Happy Path" aligné à gauche)
```

1.2 L'interface native `error` : Une simplicité absolue

En Go, `error` n'est pas une classe magique du compilateur : c'est une **simple interface pré-déclarée** dans l'univers global du langage (exactement comme `any`), qui ne contient qu'une seule méthode :

```go
type error interface {
	Error() string
}
```

Conséquence directe (Acquis de J2S2) : N'importe quelle `struct` (ou n'importe quel type) qui implémente une méthode `Error() string` est automatiquement une `error` valide en Go !

- Lorsque tout s'est bien passé, la fonction retourne `nil` pour l'erreur.
- Pour créer une erreur textuelle simple à la volée, la bibliothèque standard propose deux fonctions :
  - `errors.New("message statique")` (package `"errors"`)
  - `fmt.Errorf("message formaté pour l'ID %d", id)` (package `"fmt"`)

Convention de style officielle de Go : Les messages d'erreur en Go **ne commencent jamais par une majuscule** et **ne se terminent jamais par un point** (ni par `\n`), car ils ont vocation à être concaténés les uns aux autres lors de la remontée de la pile d'appels (ex: `"ouverture config: lecture fichier: permission refusée"`).

---

2. Les 3 niveaux de modélisation des erreurs en Go

Selon le besoin de l'appelant, Go distingue trois stratégies pour créer et exposer des erreurs.

2.1 Niveau 1 : Les Erreurs Sentinelles (*Sentinel Errors*)

Lorsque l'appelant a besoin de savoir *quel cas précis* s'est produit pour adapter son comportement (ex: fin de fichier atteinte `io.EOF`, ou aucun enregistrement SQL trouvé `sql.ErrNoRows`), on déclare une variable d'erreur publique au niveau du package. Par convention stricte en Go, son nom commence toujours par le préfixe **`Err`** :

```go
package coffre

import "errors"

// Erreurs sentinelles exportées (Convention : préfixe 'Err')
var (
	ErrSoldeInsuffisant = errors.New("solde insuffisant pour ce retrait")
	ErrCompteBloque     = errors.New("le compte est temporairement gelé")
)

func Retirer(solde, montant float64) (float64, error) {
```

```
	if montant > solde {
		return solde, ErrSoldeInsuffisant
	}
	return solde - montant, nil
}
```

2.2 Niveau 2 : Les Erreurs Typées Personnalisées (*Custom Error Structs*)

Une erreur sentinelle est statique : elle ne peut pas transporter de données dynamiques (quel était le solde manquant ? quelle ligne du fichier JSON a échoué ? quel est le code statut HTTP ?). Pour transporter du contexte structuré, on crée sa propre **`struct`** (par convention suffixée par **`Error`**) qui implémente la méthode `Error() string` :

```
Go
package reseau

import "fmt"

// Structure d'erreur personnalisée (Convention : suffixe 'Error')
type RequeteHTTPError struct {
	URL        string
	CodeStatut int
	Retentable bool
}

// Implémentation implicite de l'interface 'error' (sur Pointer Receiver
recommandé)
func (e *RequeteHTTPError) Error() string {
	return fmt.Sprintf("échec HTTP %d sur %s (retentable=%v)", e.CodeStatut,
e.URL, e.Retentable)
}

func AppelerAPI(url string) error {
	// Retourne un pointeur vers notre struct d'erreur :
	return &RequeteHTTPError{
		URL:        url,
		CodeStatut: 503,
		Retentable: true,
	}
}
```

---

3. L'*Error Wrapping* (Enrichissement) et l'Inspection (Go 1.13+)

3.1 Le problème de la perte de contexte vs perte de cause

En entreprise, une erreur traverse plusieurs couches (Base de données → Repository → Service métier → Contrôleur HTTP) :

- Si vous faites simplement `return err` à chaque couche, vous recevez dans les logs `"connection refused"` sans aucune idée de quelle fonction métier ni de quel client il s'agissait.
- Si vous écriviez `return fmt.Errorf("échec facturation client %d: %v", id, err)` (avec `%v`), vous transformiez l'erreur originale en une nouvelle chaîne de texte plate : **l'erreur originale était détruite** et l'appelant ne pouvait plus vérifier s'il s'agissait de `sql.ErrNoRows` !

3.2 La solution : Le verbe `%w` (*Wrap*) et la méthode `Unwrap() error`

Depuis Go 1.13, `fmt.Errorf` supporte le verbe spécial **`%w`** (*Wrap* / Envelopper). Au lieu d'écraser l'erreur d'origine en simple texte, `%w` crée une erreur englobante qui **conserve une référence cachée vers l'erreur d'origine** (telle une poupée russe / liste chaînée d'erreurs) :

```Go
func ChargerProfilClient(id int) error {
	err := requeteSQL(id)
	if err != nil {
		// %w ajoute le contexte métier TOUT EN CONSERVANT l'erreur 'err' intacte à l'intérieur !
		return fmt.Errorf("impossible de charger le profil %d: %w", id, err)
	}
	return nil
}
```

Sous le capot : Toute structure d'erreur personnalisée peut également participer à cette chaîne en implémentant la méthode `Unwrap() error` qui retourne l'erreur sous-jacente (`Cause error`).

3.3 Inspecter la chaîne d'erreurs : `errors.Is` et `errors.As`

Puisque les erreurs sont désormais enveloppées dans une ou plusieurs couches de `%w`, **il ne faut plus jamais utiliser `if err == ErrSoldeInsuffisant` ni un Type Assertion direct `err.(*RequeteHTTPError)`** (car ils ne regardent que la couche extérieure de la poupée russe !).

Go fournit deux fonctions qui déroulent automatiquement toute la chaîne d'erreurs (`Unwrap`) jusqu'au bout :

1. `errors.Is(err, cible)` → Remplace `==` pour les Erreurs Sentinelles

Vérifie si **au moins un maillon** de la chaîne d'erreurs `err` est égal à l'erreur sentinelle `cible` :

```
Go
err := EffectuerViremementComplexe()
if err != nil {
	// Déroule toutes les couches de '%w' pour chercher 'ErrSoldeInsuffisant' :
	if errors.Is(err, ErrSoldeInsuffisant) {
		fmt.Println("Action : Proposer un découvert autorisé au client.")
	}
}
```

2. `errors.As(err, &ptrCible)` → Remplace le Type Assertion pour les Structs d'erreurs

Vérifie si **au moins un maillon** de la chaîne d'erreurs est du type de la structure cible, et si oui, **extrait automatiquement ses champs** dans le pointeur passé en second argument :

```
Go
err := SynchroniserCatalogue()
if err != nil {
	var httpErr *RequeteHTTPError // Déclaration d'un pointeur du type recherché

	// Attention : on passe bien l'adresse du pointeur '&httpErr' !
	if errors.As(err, &httpErr) {
		fmt.Printf("Code HTTP détecté en profondeur : %d sur %s\n",
httpErr.CodeStatut, httpErr.URL)
		if httpErr.Retentable {
			fmt.Println("Planification d'une nouvelle tentative dans 5
secondes...")
		}
	}
}
```

---

4. Bonnes pratiques d'architecture sur les erreurs

1. **Règle d'or : *"Handle errors once"* (Ne gérez une erreur qu'une seule fois) :** Le pire anti-pattern en Go est de faire `log.Println(err)` **puis** `return err` à chaque niveau de fonction. Vous obtiendrez 5 lignes de logs identiques pour une seule panne !

    - **Soit** vous remontez l'erreur en l'enrichissant avec du contexte (`return fmt.Errorf("contexte: %w", err)`), **sans rien logger**.
    - **Soit** vous êtes au sommet de la pile (ex: dans `main()` ou dans le contrôleur HTTP final) : vous logguez l'erreur complète une seule fois et vous prenez la décision finale (renvoyer une erreur HTTP 500, utiliser une valeur de repli...).
2. **Quand utiliser `%v` au lieu de `%w` dans `fmt.Errorf` ?** Utilisez `%w` lorsque vous voulez que l'appelant puisse inspecter l'erreur technique sous-jacente avec `errors.Is/As`. Si au contraire vous développez une librairie et que vous voulez **masquer les détails d'implémentation internes** (ex: empêcher l'appelant de se coupler à votre driver SQL interne), utilisez `%v` : le message texte sera conservé pour les logs, mais la chaîne `Unwrap` sera volontairement coupée.